

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ
к лабораторной работе № 2
на тему «Симметричная криптография. Стандарт шифрования СТБ
34.101.31-2011»

Выполнил

Е. С. Кахновский

Проверил

Е. А. Лещенко

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	15
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является реализация программных средств шифрования и дешифрования текстовых файлов при помощи стандарта шифрования СТБ 34.101.31-2011 в режиме простой замены.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

BelT — государственный стандарт симметричного шифрования и контроля целостности Республики Беларусь. Полное название стандарта — СТБ 34.101.31-2007 «Информационные технологии и безопасность. Криптографические алгоритмы шифрования и контроля целостности». Принят в качестве предварительного стандарта в 2007 году. Введён в действие в качестве окончательного стандарта в 2011 году.

BelT — блочный шифр с 256-битным ключом и 8 циклами криптопреобразований, оперирующий с 128-битными словами. Криптографические алгоритмы стандарта построены на основе базовых режимов шифрования блоков данных. Все алгоритмы стандарта делятся на 8 групп:

- алгоритмы шифрования в режиме простой замены;
- алгоритмы шифрования в режиме сцепления блоков;
- алгоритмы шифрования в режиме гаммирования с обратной связью;
- алгоритмы шифрования в режиме счётчика;
- алгоритм выработки имитовставки ;
- алгоритмы одновременного шифрования и имитозащиты данных;
- алгоритмы одновременного шифрования и имитозащиты ключей;
- алгоритм хеширования;

Первые четыре группы предназначены для обеспечения безопасного обмена сообщениями. Каждая группа включает алгоритм шифрования и алгоритм расшифрования на секретном ключе.

Стороны, располагающие общим ключом, могут организовать обмен сообщениями путём их шифрования перед отправкой и расшифрования после получения.

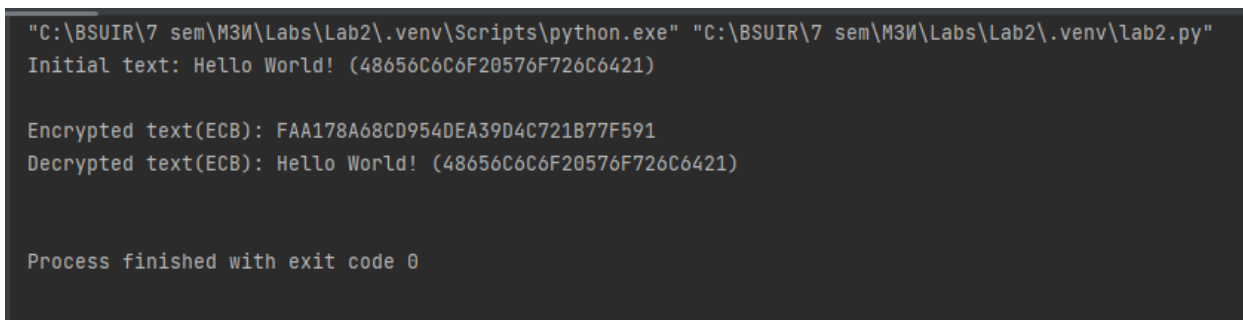
В режимах простой замены и сцепления блоков шифруются сообщения, которые содержат хотя бы один блок, а в режимах гаммирования с обратной связью и счётчика — сообщения произвольной длины.

Пятый алгоритм предназначен для контроля целостности сообщений с помощью имитовставок — контрольных слов, которые определяются с использованием секретного ключа. Стороны, располагающие общим ключом, могут организовать контроль целостности при обмене сообщениями путём добавления к ним имитовставок при отправке и проверки имитовставок при получении. Проверка имитовставок дополнительно позволяет стороне получателю убедиться в знании стороной-отправителем секретного ключа, то есть проверить подлинность сообщений.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

В ходе выполнения лабораторной было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования СТБ 34.101.31-2011 в режиме простой замены.

Начальный текст находится в файле text.txt. После шифрования зашифрованная информация помещается в консоль, после чего она снова дешифруется и помещается в консоль. Результат работы программы представлен на рисунке 3.1.

A screenshot of a terminal window with a dark background and light-colored text. The text shows the execution of a Python script. It starts with the command path, followed by the initial text 'Hello World!' and its hexadecimal representation. Then, it shows the encrypted text (ECB) as a long hexadecimal string. Next, it shows the decrypted text (ECB) as 'Hello World!' with its hexadecimal representation. Finally, it shows the process finished with exit code 0.

```
"C:\BSUIR\7 sem\МЗИ\Labs\Lab2\.venv\Scripts\python.exe" "C:\BSUIR\7 sem\МЗИ\Labs\Lab2\.venv\lab2.py"
Initial text: Hello World! (48656C6C6F20576F726C6421)

Encrypted text(ECB): FAA178A68CD954DEA39D4C721B77F591
Decrypted text(ECB): Hello World! (48656C6C6F20576F726C6421)

Process finished with exit code 0
```

Рисунок 3.1 – Результат работы программы

Таким образом, в ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования СТБ 34.101.31-2011 в режиме простой замены.

ВЫВОДЫ

В ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования СТБ 34.101.31-2011 в режиме простой замены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Алгоритм шифрования СТБ 34.101.31-2011 [Электронный ресурс]. – Режим доступа: <https://apmi.bsu.by/assets/files/std/belt-spec27.pdf> – Дата доступа: 02.09.2024.

[2] Реализация алгоритма шифрования простой замены [Электронный ресурс]. – Режим доступа: <https://www.cyberforum.ru/thread1109400.html>. – Дата доступа: 02.09.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

Листинг 1 – Исходный код

```
import random
import string

    return U_32(int(u, 2) - int(v, 2))

def xor_32(u, v):
    return '{0:b}'.format(int(u, 2) ^ int(v, 2)).zfill(32)

def xor(u, v):
    return '{0:b}'.format(int(u, 2) ^ int(v, 2)).zfill(128)

def U(number):
    return '{0:b}'.format(int(number % 2 ** 128)).zfill(128)

def plus(u, v):
    return U(int(u, 2) + int(v, 2))

def encrypt_block(block, key):
    X = list(chunks(block, 32))
    theta = list(chunks(key, 32))
    K = []
    for i in range(56):
        K.append(theta[i % len(theta)])

    a, b, c, d = X[0], X[1], X[2], X[3]
    for i in range(1, 9):
        b = xor_32(b, G(plus_32(a, K[7 * i - 7]), 5))
        c = xor_32(c, G(plus_32(d, K[7 * i - 6]), 21))
        a = minus_32(a, G(plus_32(b, K[7 * i - 5]), 13))
        e = xor_32(G(plus_32(b, plus_32(c, K[7 * i - 4])), 21), U_32(i))
        b = plus_32(b, e)
        c = minus_32(c, e)
        d = plus_32(d, G(plus_32(c, K[7 * i - 3]), 13))
        b = xor_32(b, G(plus_32(a, K[7 * i - 2]), 21))
        c = xor_32(c, G(plus_32(d, K[7 * i - 1]), 5))
        a, b = b, a
        c, d = d, c
        b, c = c, b

    return b + d + a + c

def decrypt_block(block, key):
    X = list(chunks(block, 32))
    theta = list(chunks(key, 32))
    K = []
    for i in range(56):
        K.append(theta[i % len(theta)])

    a, b, c, d = X[0], X[1], X[2], X[3]
    for i in range(8, 0, -1):
```

```

        b = xor_32(b, G(plus_32(a, K[7 * i - 1]), 5))
        c = xor_32(c, G(plus_32(d, K[7 * i - 2]), 21))
        a = minus_32(a, G(plus_32(b, K[7 * i - 3]), 13))
        e = xor_32(G(plus_32(b, plus_32(c, K[7 * i - 4])), 21), U_32(i))

    return c + a + d + b

def chunks(string, size):
    for i in range(0, len(string), size):
        yield string[i:i + size]

def get_padding(text):
    padding_len = 16 - ((len(text) // 8) % 16)
    return padding_len * '{0:b}'.format(padding_len).zfill(8)

def remove_padding(text):
    padding_len = int(text[-8:], 2)
    return text[:-padding_len * 8]

def ecb_encrypt(text, key):
    text += get_padding(text)
    X = list(chunks(text, 128))

    encrypted_parts = []

    for block in X:
        encrypted_parts.append(encrypt_block(block, key))

    return ''.join(encrypted_parts)

def ecb_decrypt(text, key):
    X = list(chunks(text, 128))

    decrypted_parts = []

    for block in X:
        decrypted_parts.append(decrypt_block(block, key))

    return bin_to_str(remove_padding(''.join(decrypted_parts)))

if __name__ == '__main__':
    with open('text.txt', 'r') as file:
        t = file.read()

    k_x = '12345678987654321234567898765432'

    print("Initial text: {} ({}).format(t, str_to_hex(t))
    print()

    e = ecb_encrypt(str_to_bin(t), str_to_bin(k_x))
    print("Encrypted text (ECB): {}".format(bin_to_hex(e)))
    d = ecb_decrypt(e, str_to_bin(k_x))
    print("Decrypted text (ECB): {} ({}).format(d, str_to_hex(d))
    print()

```